UCTMA Phase 1 Teknik Dokümantasyonu

1. Dokümanın amacı

Bu doküman UCTMA projesinin Phase 1 sürümünü açıklar.

Doküman aşağıdaki bileşenleri kapsar:

- Backend
- Frontend
- Docling Converter Service
- RabbitMQ
- PostgreSQL
- Docker Compose
- Dosya depolama
- Authentication ve Authorization
- API endpoint'leri
- Hata yönetimi
- Kurulum ve bakım

Bu dokümanda kullanılan bilgiler proje kaynak kodundan ve Compose dosyalarından alınmıştır.

2. Sistem özeti

UCTMA, desteklenen dosyaları Markdown formatına dönüştürür.

İsteğe bağlı olarak Markdown içeriğini chunk olarak da üretir.

Dosya işleme asenkron yapılır.

İş akışı aşağıdaki gibidir:

1. Kullanıcı Frontend üzerinden login olur.
2. Kullanıcı bir dosya seçer.
3. Frontend dosyayı Backend'e gönderir.
4. Backend dosyayı /uploads altında saklar.
5. Backend dosya kaydını PostgreSQL'e ekler.
6. Backend RabbitMQ'ya bir iş mesajı gönderir.
7. Docling Converter Service mesajı RabbitMQ'dan alır.
8. Converter dosyayı ortak /uploads volume'ünden okur.
9. Converter dosyayı Markdown'a dönüştürür.
10. Converter, istenmişse chunk üretir.
11. Converter sonucu Backend callback endpoint'ine gönderir.
12. Backend Markdown ve chunk kayıtlarını saklar.
13. Frontend dosya listesini yeniler.

3. Bileşenler

3.1 Backend

Backend aşağıdaki teknolojileri kullanır:

- Java 21
- Spring Boot 4.2.0-M2
- Spring Web MVC
- Spring Data JPA
- Spring Security
- Spring Validation
- Spring AMQP RabbitMQ
- PostgreSQL Driver
- JJWT
- Springdoc OpenAPI
- Spring Mail

Backend'in kaynak dizini:

```
uctma/src/main/java/com/app/uctma/backend
```

Backend'in container içindeki HTTP portu 8080'dir.

Backend Docker image'i Maven ile build edilir.

Dockerfile, Maven build aşamasında Java 21 kullanır.

3.2 Frontend

Frontend aşağıdaki teknolojileri kullanır:

- Node.js 20 Alpine image
- Next.js 14.2.15
- React 18.3.1
- React DOM 18.3.1
- TypeScript 5.7.2
- Tailwind CSS 3.4.16

Frontend container portu 3000'dür.

Frontend, Backend'e doğrudan tarayıcı isteği göndermez.

Frontend /api/* isteklerini Next.js rewrite kuralı ile Backend'e yönlendirir.

Rewrite hedefi aşağıdaki biçimdedir:

```
${BACKEND_URL}/uctma/api/*
```

3.3 Docling Converter Service

Converter Python 3.11-slim image'i üzerinde çalışır.

Converter container aynı anda iki işlem başlatır:

1. FastAPI HTTP server

2. RabbitMQ consumer

FastAPI server 8000 portunda çalışır.

RabbitMQ consumer file_processing_queue kuyruğunu dinler.

Converter Dockerfile'daki başlangıç komutu aşağıdaki iki işlemi paralel başlatır:

```
uvicorn main:app --host 0.0.0.0 --port 8000
python consumer.py
```

Converter model cache için docling_models volume'ünü kullanır.

3.4 RabbitMQ

RabbitMQ şu image ile çalışır:

```
rabbitmq:3-management-alpine
```

RabbitMQ container adı:

```
uctma_rabbitmq
```

RabbitMQ AMQP portu 5672'dir.

RabbitMQ Management portu 15672'dır.

RabbitMQ verileri rabbitmq_data volume'ünde tutulur.

Backend aşağıdaki kaynakları tanımlar:

- Queue: file_processing_queue
- Exchange: file_processing_exchange
- Routing key: file_processing_routing_key

Queue durable olarak tanımlanır.

Exchange DirectExchange olarak tanımlanır.

Backend queue ile exchange arasında binding oluşturur.

Converter consumer queue'yu durable olarak tanımlar.

Converter prefetch_count=1 kullanır.

Bu ayar aynı anda bir mesaj işlenmesini sağlar.

3.5 PostgreSQL

PostgreSQL şu image ile çalışır:

```
postgres:15-alpine
```

PostgreSQL container adı:

```
rag-platform-postgres
```

PostgreSQL database adı .env içindeki DB_NAME değerinden alınır.

Mevcut yapılandırmada database adı rag_database'dir.

PostgreSQL kullanıcı adı .env içindeki DB_USERNAME değerinden alınır.

Mevcut yapılandırmada kullanıcı adı rag_admin'dir.

PostgreSQL container portu 5432'dir.

Host portu DB_HOST_PORT ile belirlenir.

Mevcut yapılandırmada host portu 5433'tür.

PostgreSQL verileri pgdata volume'ünde tutulur.

Backend PostgreSQL'e container network üzerinden aşağıdaki biçimde bağlanır:

```
jdbc:postgresql://postgres-db:${DB_CONTAINER_PORT}/${DB_NAME}
```

Backend'te şu ayar kullanılır:

```
spring.jpa.hibernate.ddl-auto=update
```

Bu nedenle Hibernate mevcut entity modeline göre database şemasını güncellemeyi dener.

Projede ayrı bir migration aracı veya migration script'i bulunmamaktadır.

4. Docker network ve volume yapısı

Backend Compose ve Converter Compose aşağıdaki external network adını kullanır:

```
uctma_default
```

Frontend Compose bu network'ü external olarak kullanır.

Backend ve Converter aşağıdaki ortak volume'ü kullanır:

```
uctma_uploads
```

Backend bu volume'ü /uploads olarak mount eder.

Converter bu volume'ü /uploads olarak mount eder.

Bu ortak volume aşağıdaki dosyaları içerir:

- Orijinal yüklenen dosyalar.
- Üretilen Markdown dosyaları.
- Üretilen chunk dosyaları.

Converter ayrıca model cache için şu volume'ü kullanır:

```
docling_models
```

PostgreSQL volume'ü:

```
pgdata
```

RabbitMQ volume'ü:

```
rabbitmq_data
```

5. Ortam değişkenleri

5.1 Backend ortam değişkenleri

Backend uctma/.env dosyasındaki değerleri kullanır.

Başlıca değişkenler:

Değişken	Kullanım
DB_NAME	PostgreSQL database adı
DB_USERNAME	PostgreSQL kullanıcı adı
DB_PASSWORD	PostgreSQL şifresi
DB_CONTAINER_PORT	PostgreSQL container portu
DB_HOST_PORT	PostgreSQL host portu
JWT_SECRET	JWT imzalama anahtarı
JWT_EXPIRATION	JWT geçerlilik süresi
SMTP_USERNAME	E-posta hesabı
SMTP_PASSWORD	E-posta hesabı şifresi
SMTP_HOST	SMTP sunucusu
SMTP_PORT	SMTP portu
UPLOAD_PATH	Dosya saklama dizini
RABBITMQ_HOST	RabbitMQ host adı
RABBITMQ_PORT	RabbitMQ AMQP portu
RABBITMQ_USER	RabbitMQ kullanıcı adı
RABBITMQ_PASS	RabbitMQ şifresi
RABBITMQ_QUEUE_FILE	Dosya işleme queue adı
RABBITMQ_EXCHANGE_FILE	Dosya işleme exchange adı
RABBITMQ_ROUTING_KEY_FILE	Dosya işleme routing key değeri

Secret değerleri bu dokümana yazılmamalıdır.

Üretim ortamında varsayılan guest bilgileri kullanılmamalıdır.

5.2 Converter ortam değişkenleri

Converter docling_converter_service/.env dosyasını kullanır.

Başlıca değişkenler:

Değişken	Kullanım
RABBITMQ_HOST	RabbitMQ host adı
RABBITMQ_PORT	RabbitMQ AMQP portu
RABBITMQ_USER	RabbitMQ kullanıcı adı
RABBITMQ_PASS	RabbitMQ şifresi
RABBITMQ_QUEUE_FILE	Dinlenecek queue adı
RABBITMQ_EXCHANGE_FILE	Ortam uyumluluğu için exchange adı
RABBITMQ_ROUTING_KEY_FILE	Ortam uyumluluğu için routing key değeri
BACKEND_CALLBACK_URL	Backend callback adresi

Mevcut callback adresi:

```
http://rag-platform-backend:8080/uctma/api/file/callback
```

5.3 Frontend ortam değişkeni

Frontend BACKEND_URL değişkenini kullanır.

Mevcut değer:

```
http://backend-service:8080
```

Frontend rewrite bu değerin sonuna /uctma/api ekler.

6. Backend database modeli

6.1 users tablosu

User entity'si users tablosuna karşılık gelir.

Alanlar:

- id
- email
- password
- role
- newPasswordFlag

Bir user birden fazla FileMetadata kaydına sahip olabilir.

email alanı unique'tir.

6.2 files tablosu

FileMetadata entity'si files tablosuna karşılık gelir.

Alanlar:

- id
- originalFileName
- filePath
- mdFilePath
- fileType
- status
- maxTokens
- user_id
- createdAt

Bir file bir user'a bağlıdır.

Bir file birden fazla FileChunk kaydına sahip olabilir.

6.3 file_chunks tablosu

FileChunk entity'si file_chunks tablosuna karşılık gelir.

Alanlar:

- id
- chunkIndex
- chunkName
- content
- tokenCount
- file_id
- createdAt
- filePath

Chunk kaydı bir file'a bağlıdır.

File silindiğinde entity cascade ayarı database ilişkisini de etkiler.

Backend ayrıca chunk dosyasını fiziksel olarak siler.

7. Authentication ve Authorization

Backend stateless Authentication kullanır.

Authentication JWT ile yapılır.

Session database üzerinde tutulmaz.

Korumalı isteklerde aşağıdaki header gönderilir:

```
Authorization: Bearer <JWT>
```

Desteklenen role değerleri:

- ROLE_USER
- ROLE_ADMIN

ROLE_ADMIN aşağıdaki endpoint'lere erişebilir:

- User register
- User list
- User delete

Giriş yapmış her kullanıcı aşağıdaki işlemleri yapabilir:

- Password change
- Kendi file kayıtlarını listeleme
- Kendi file'ını yükleme
- Kendi file'ını silme
- Kendi file'ını yeniden dönüştürme
- Kendi file'ının chunk kayıtlarını listeleme
- Kendi file'ının çıktılarını indirme
- Kendi chunk kaydını silme

CheckAccess sınıfı file ve chunk sahipliğini kontrol eder.

Bir kullanıcı başka kullanıcıya ait file veya chunk'a erişemez.

Login endpoint'i ve converter callback endpoint'i public'tir.

Swagger endpoint'leri de public olarak yapılandırılmıştır:

- /v3/api-docs/**
- /swagger-ui/**
- /swagger-ui.html

8. User işlemleri

8.1 User oluşturma

Admin aşağıdaki endpoint'i çağırır:

```
POST /uctma/api/auth/register
```

Request body:

```
{
"email": "user@example.com",
"role": "ROLE_USER"
}
```

Backend geçici bir şifre üretir.

Backend şifreyi encode ederek database'e kaydeder.

Backend geçici şifreyi e-posta ile gönderir.

Yeni user için newPasswordFlag=true olur.

8.2 Login

Endpoint:

```
POST /uctma/api/auth/login
```

Login response aşağıdaki alanları içerir:

- email
- token
- tokenType
- newPasswordRequired
- role

Frontend token ve role değerlerini localStorage içinde tutar.

Kullanılan key'ler:

- uctma_token
- uctma_role

8.3 İlk şifre değişikliği

Geçici şifre ile login yapıldığında newPasswordRequired=true olabilir.

Frontend bu durumda password formunu açar.

Endpoint:

```
POST /uctma/api/auth/change-password
```

Request body:

```
{
"currentPassword": "geçici-şifre",
"newPassword": "yeni-şifre",
"confirmNewPassword": "yeni-şifre"
}
```

Başarılı işlemden sonra newPasswordFlag=false olur.

8.4 User silme

Endpoint:

```
DELETE /uctma/api/auth/delete?email=user@example.com
```

Aktif admin kendi hesabını silemez.

9. File işlemleri

9.1 Desteklenen uzantılar

Backend ve Converter aşağıdaki uzantıları desteklenen uzantı olarak tanımlar:

- .pdf
- .docx
- .pptx
- .html
- .csv
- .xlsx
- .txt
- .md

Backend dosya adının son uzantısını kontrol eder.

Örnek:

```
report.pdf
report.py.pdf
```

İki dosyada da son uzantı .pdf olarak değerlendirilir.

Dosya içeriği uzantı ile uyumlu olmalıdır.

Bir Python dosyasının adını .pdf yapmak dosyayı PDF'e dönüştürmez.

Frontend desteklenmeyen uzantıyı queue işleminden önce reddeder.

Backend de aynı uzantı kontrolünü tekrar yapar.

9.2 File upload

Endpoint:

```
POST /uctma/api/file/upload
```

İstek multipart/form-data olmalıdır.

Form alanları:

- file
- conversionType
- maxTokens

conversionType değerleri:

- CONVERT
- CONVERT_AND_CHUNK

Varsayılan conversionType değeri CONVERT'dür.

Varsayılan maxTokens değeri 400'dür.

Spring Multipart limitleri:

- Maksimum file size: 50MB
- Maksimum request size: 50MB

Backend upload sırasında aşağıdaki işlemleri yapar:

1. Login olmuş user'ı bulur.
2. Dosya uzantısını kontrol eder.
3. Dosyayı UPLOAD_PATH altında saklar.
4. PENDING status ile file metadata oluşturur.
5. RabbitMQ'ya mesaj gönderir.
6. File bilgilerini response olarak döndürür.

Upload response alanları:

- id
- originalFileName
- status
- maxTokens
- createdAt

9.3 File listeleme

Endpoint:

```
GET /uctma/api/file
```

Endpoint yalnızca Authentication yapan user'ın file kayıtlarını döndürür.

Response içinde aşağıdaki alanlar bulunur:

- id
- originalFileName
- status
- maxTokens
- createdAt

9.4 Yeniden dönüştürme

Endpoint:

```
POST /uctma/api/file/{fileId}/convert
```

Query parametreleri:

- conversionType
- maxTokens

conversionType gönderilmezse CONVERT kullanılır.

maxTokens gönderilmezse mevcut file değerinin kullanılması beklenir.

Backend yeniden dönüştürme sırasında:

1. File sahipliğini kontrol eder.
2. Orijinal file uzantısını tekrar kontrol eder.
3. Eski chunk dosyalarını siler.
4. Eski chunk database kayıtlarını siler.
5. Eski Markdown dosyasını siler.
6. mdFilePath değerini temizler.
7. File status değerini PENDING yapar.
8. Yeni RabbitMQ mesajı gönderir.

Orijinal file silinmez.

9.5 File silme

Endpoint:

```
DELETE /uctma/api/file/{fileId}
```

Backend aşağıdaki verileri siler:

- Chunk fiziksel dosyaları
- Chunk database kayıtları
- Markdown fiziksel dosyası
- Orijinal fiziksel dosya
- File metadata kaydı

9.6 Orijinal file indirme

Endpoint:

```
GET /uctma/api/file/{fileId}/download
```

Bu endpoint original file'ı binary response olarak döndürür.

9.7 Markdown indirme

Endpoint:

```
GET /uctma/api/file/{fileId}/download-md
```

Markdown dosyası henüz oluşturulmamışsa Backend mdFilePath değerini boş bulur.

Bu durumda indirme işlemi için kullanılabilir bir Resource bulunmayabilir.

Frontend bu endpoint'i tamamlanmış output için kullanmalıdır.

10. Chunk işlemleri

10.1 Chunk üretimi

Chunk üretimi yalnızca CONVERT_AND_CHUNK seçildiğinde istenir.

Docling dosyalarında HybridChunker kullanılır.

HybridChunker için maxTokens değeri kullanılır.

Plain text dosyalarında word-based chunking kullanılır.

Plain text chunking şu uzantılarda kullanılır:

- .csv
- .txt
- .md

Plain text chunk size yaklaşık bir değerdir.

Bu değer gerçek tokenizer sonucu değildir.

10.2 Chunk listeleme

Endpoint:

```
GET /uctma/api/file/{fileId}/chunks
```

Response her chunk için şu alanları döndürür:

- id
- chunkName

Chunk kayıtları chunkIndex sırasına göre listelenir.

10.3 Tek chunk indirme

Endpoint:

```
GET /uctma/api/file/chunks/{chunkId}/download
```

Response text/plain olarak döndürülür.

10.4 Tüm chunk'ları ZIP olarak indirme

Endpoint:

```
GET /uctma/api/file/chunks/{fileId}/download-all
```

Response application/zip olarak döndürülür.

10.5 Chunk silme

Endpoint:

```
DELETE /uctma/api/file/chunks/{chunkId}
```

Backend önce chunk sahipliğini kontrol eder.

Sonra fiziksel chunk dosyasını ve database kaydını siler.

11. RabbitMQ mesaj formatı

Backend'in gönderdiği mesaj FileMessageDTO alanlarını kullanır:

```
{
"fileId": 10,
"filePath": "/uploads/example.pdf",
"originalFileName": "example.pdf",
"conversionType": "CONVERT_AND_CHUNK",
"maxTokens": 400
}
```

Converter bu alanları kullanır:

- fileId
- filePath
- conversionType
- maxTokens

Converter filePath ile ortak /uploads volume'ündeki file'ı okur.

Consumer prefetch_count=1 ile çalışır.

12. Converter işleme davranışı

12.1 Docling dosyaları

.pdf, .docx, .pptx, .html ve .xlsx dosyaları Docling'e gönderilir.

Docling conversion sonucundan Markdown export edilir.

CONVERT işleminde chunk üretilmez.

CONVERT_AND_CHUNK işleminde HybridChunker kullanılır.

Chunk response alanları:

- chunkIndex
- text
- pageNumbers

12.2 Plain text dosyaları

.csv, .txt ve .md dosyaları UTF-8 text olarak okunur.

Bu dosyalar için Docling conversion çağrısı yapılmaz.

CONVERT işleminde okunan text convertedText olur.

CONVERT_AND_CHUNK işleminde text word-based chunk'lara ayrılır.

Plain text chunk'larında pageNumbers boş list olarak gönderilir.

12.3 Dosya yoksa

Converter file path'in varlığını kontrol eder.

File yoksa processing error oluşur.

12.4 Desteklenmeyen uzantı

Converter FastAPI endpoint'leri son uzantıyı kontrol eder.

RabbitMQ consumer doğrudan process_file_with_docling fonksiyonunu çağırır.

Backend upload öncesinde desteklenmeyen uzantıyı engeller.

Consumer'a doğrudan desteklenmeyen bir file mesajı gönderilirse processing veya Docling hatası oluşabilir.

Bu hata callback ile FAILED status olarak Backend'e bildirilebilir.

13. Callback iş akışı

Callback endpoint'i:

```
POST /uctma/api/file/callback
```

Bu endpoint Frontend tarafından çağrılmaz.

Bu endpoint yalnızca Converter'ın Backend'e sonucu bildirmesi için kullanılır.

Callback request alanları:

- fileId
- status
- convertedText
- chunks
- errorMessage

Başarılı callback örneği:

```
{
"fileId": 10,
"status": "COMPLETED",
"convertedText": "# Başlık",
"chunks": [],
"errorMessage": null
}
```

Hatalı callback örneği:

```
{
"fileId": 10,
"status": "FAILED",
"convertedText": null,
"chunks": [],
"errorMessage": "Dosya formatı desteklenmiyor"
}
```

Backend callback aldığında:

1. fileId ile file metadata kaydını bulur.

2. File status değerini callback status değerine ayarlar.

3. convertedText boş değilse Markdown dosyası kaydeder.

4. Chunk listesi boş değilse chunk dosyalarını ve database kayıtlarını oluşturur.

5. Status FAILED ise error mesajını loglar.

Callback endpoint'i SecurityConfig içinde public'tir.

JWT filter callback path'ini filtrelemez.

14. Consumer acknowledgement davranışı

Converter callback gönderirken aşağıdaki davranışı kullanır:

- Başarılı callback sonrası message ack edilir.
- Kalıcı HTTP 4xx callback hatası sonrası message ack edilir.
- Geçici network veya Backend bağlantı hatası sonrası message nack ile requeue edilir.

Bu davranış geçersiz bir message'ın sonsuz tekrar işlenmesini engellemeyi amaçlar.

RabbitMQ bağlantısı kapanırsa consumer beş saniye bekler ve yeniden bağlanmayı dener.

RabbitMQ bağlantısında heartbeat değeri varsayılan olarak 600 saniyedir.

15. File status davranışı

Backend enum içinde şu status değerleri vardır:

- PENDING
- PROCESSING
- COMPLETED
- FAILED

Upload ve reconvert sırasında Backend status değerini PENDING yapar.

Mevcut Backend upload ve callback kodunda PROCESSING değerini set eden bir işlem bulunmamaktadır.

Bu nedenle PROCESSING değeri modelde tanımlı olsa da mevcut normal akışta görülmeyebilir.

Callback başarılıysa Converter COMPLETED gönderir.

Converter işlem hatasında FAILED gönderir.

Frontend bu status değerlerini aşağıdaki Türkçe etiketlerle gösterir:

- Bekliyor
- İşleniyor
- Tamamlandı
- Başarısız

Frontend file listesini periyodik olarak yeniler.

16. Frontend mimarisi

Frontend dosya yapısı:

```
uctma_frontend/
 app/
  page.tsx
  layout.tsx
  globals.css
 components/
  ChunksDialog.tsx
  FileTable.tsx
  FileUploader.tsx
  LoginScreen.tsx
  Modal.tsx
  Navbar.tsx
  PasswordForm.tsx
  RegisterForm.tsx
  UsersDialog.tsx
```

```
hooks/
 useAuth.ts
 useFiles.ts
 useTheme.ts
lib/
api.ts
types.ts
```

app/page.tsx ekran state'ini ve bileşenler arası akışı yönetir.

components/ UI bileşenlerini içerir.

hooks/useAuth.ts Authentication state'ini yönetir.

hooks/useFiles.ts file, chunk ve user API çağrılarını yönetir.

hooks/useTheme.ts theme state'ini yönetir.

lib/api.ts JSON ve binary API çağrıları için yardımcı fonksiyonlar içerir.

lib/types.ts Backend response tiplerini içerir.

16.1 Theme

Tailwind darkMode: "class" ayarıyla kullanılır.

Theme seçimi uctma_theme localStorage key'i ile saklanır.

Frontend light ve dark theme toggle içerir.

16.2 Frontend API davranışı

JSON API çağrıları /api prefix'i ile yapılır.

Korumalı isteklerde Authorization header'ı eklenir.

Download isteklerinde response blob olarak alınır.

JSON response'larda Backend'in ApiResponse zarfı kontrol edilir.

HTTP başarısızsa veya success=false ise Frontend error gösterir.

17. REST API özeti

17.1 Authentication endpoint'leri

Method	Endpoint	Yetki	Açıklama
POST	/uctma/api/auth/login	Public	Login
POST	/uctma/api/auth/register	ROLE_ADMIN	User oluşturma
POST	/uctma/api/auth/change-password	Authenticated	Password değiştirme
GET	/uctma/api/auth/list-users	ROLE_ADMIN	User listesi
DELETE	/uctma/api/auth/delete?email=...	ROLE_ADMIN	User silme

17.2 File endpoint'leri

Method	Endpoint	Yetki	Açıklama
POST	/uctma/api/file/upload	Authenticated	File yükleme
GET	/uctma/api/file	Authenticated	User file listesi
POST	/uctma/api/file/{fileId}/convert	Authenticated	Yeniden dönüştürme
DELETE	/uctma/api/file/{fileId}	Authenticated	File silme
GET	/uctma/api/file/{fileId}/download	Authenticated	Original file indirme
GET	/uctma/api/file/{fileId}/download-md	Authenticated	Markdown indirme

Method	Endpoint	Yetki	Açıklama
GET	/uctma/api/file/{fileId}/chunks	Authenticated	Chunk listesi
DELETE	/uctma/api/file/chunks/{chunkId}	Authenticated	Chunk silme
GET	/uctma/api/file/chunks/{chunkId}/download	Authenticated	Tek chunk indirme
GET	/uctma/api/file/chunks/{fileId}/download-all	Authenticated	ZIP indirme
POST	/uctma/api/file/callback	Converter internal	İşlem sonucu callback

18. Hata yönetimi

Backend RuntimeException hatalarını 400 Bad Request response'una dönüştürür.

Validation hataları da 400 Bad Request response'u olarak döndürülür.

Backend JSON response zarfında success, data ve error alanlarını kullanır.

Frontend success=false response'larını hata olarak işler.

18.1 403 callback hatası

Callback endpoint'i kaynak kodda public'tir.

Çalışan container 403 döndürürse aşağıdakiler kontrol edilmelidir:

1. Backend image'inin güncel kaynak koddan build edilip edilmediği.

2. Converter'ın doğru Backend container adına bağlanıp bağlanmadığı.

3. uctma_default network'ünün iki Compose projesinde de aynı olup olmadığı.

4. BACKEND_CALLBACK_URL değerinin doğru olup olmadığı.

Backend'i yeniden build etmek için:

```
cd uctma
docker compose up -d --build backend-service
```

18.2 File PENDING kalırsa

Aşağıdaki loglar kontrol edilmelidir:

```
docker logs --tail 200 rag-platform-backend
docker logs --tail 200 docling_converter_service
```

Kontrol sırası:

1. RabbitMQ consumer'ın çalışıp çalışmadığı.

2. File'ın ortak /uploads volume'ünde bulunup bulunmadığı.

3. Converter callback URL'sinin doğru olup olmadığı.

4. Callback HTTP status değerinin 200 olup olmadığı.

5. Backend'in callback'i kabul edip etmediği.

18.3 Desteklenmeyen format

Dosyanın son uzantısı kontrol edilmelidir.

Dosya içeriği ile uzantı aynı formatı göstermelidir.

Sadece file adını değiştirmek format dönüşümü yapmaz.

18.4 Token length uyarısı

Docling veya model katmanında tokenizer limitini aşan içerik uyarısı görülebilir.

Bu mesaj tek başına Backend callback'inin başarısız olduğu anlamına gelmez.

Sonuç status ve callback response birlikte kontrol edilmelidir.

19. Kurulum ve çalıştırma

19.1 Ön koşullar

Gerekli araçlar:

- Docker
- Docker Compose
- Java 21 destekli Docker build ortamı
- Node.js 20 destekli Docker build ortamı

Backend ve Converter aynı Docker network'üne bağlanmalıdır.

19.2 Backend'i başlatma

```
cd uctma
docker compose up -d --build
```

Bu Compose projesi şu servisleri başlatır:

- postgres-db
- rabbitmq
- backend-service

Backend Compose içinde backend-service, postgres-db servisine depends_on ile bağlıdır.

RabbitMQ için Compose seviyesinde depends_on tanımı yoktur.

19.3 Converter'ı başlatma

```
cd docling_converter_service
docker compose up -d --build
```

Converter'ın bağlı olduğu uctma_default network'ü mevcut olmalıdır.

Converter'ın bağlı olduğu uctma_uploads volume'ü Backend ile aynı volume olmalıdır.

19.4 Frontend'i başlatma

```
cd uctma_frontend
docker compose up -d --build
```

Frontend adresi:

```
http://localhost:3000
```

Backend host adresi:

```
http://localhost:8080
```

RabbitMQ Management arayüzü:

```
http://localhost:15672
```

PostgreSQL host bağlantı portu mevcut ayarda:

```
5433
```

20. Doğrulama kontrol listesi

Kurulumdan sonra aşağıdaki kontroller yapılmalıdır:

1. PostgreSQL container çalışıyor mu?

2. RabbitMQ container çalışıyor mu?

3. RabbitMQ Management arayüzü açılıyor mu?

4. Backend 8080 portunda çalışıyor mu?

5. Converter 8000 portunda çalışıyor mu?

6. Converter /health endpoint'i cevap veriyor mu?

7. Frontend 3000 portunda açılıyor mu?

8. Login çalışıyor mu?

9. File upload sonrası queue'da mesaj oluşuyor mu?

10. Converter ortak volume'deki file'ı okuyabiliyor mu?

11. Callback 200 dönüyor mu?

12. File status COMPLETED veya FAILED oluyor mu?

Converter health endpoint'i:

```
GET http://localhost:8000/health
```

Beklenen response:

```
{
"status": "healthy",
"service": "Docling Converter Service"
}
```

21. FastAPI endpoint'leri

Converter container ayrıca aşağıdaki HTTP endpoint'lerini sağlar:

Method	Endpoint	Açıklama
GET	/health	Converter sağlık kontrolü
POST	/convert	Bir file'ı Markdown'a dönüştürme
POST	/convert-and-chunk	Bir file'ı Markdown'a dönüştürme ve chunk üretme

/convert ve /convert-and-chunk endpoint'leri UploadFile kabul eder.

Converter FastAPI endpoint'leri desteklenmeyen son uzantılar için 400 döndürür.

İşleme hataları için 500 döndürür.

RabbitMQ consumer ise bu HTTP endpoint'lerini çağırmaz.

Consumer doğrudan docling_processor.py içindeki processing fonksiyonunu çağırır.

22. Güvenlik ve işletim notları

.env dosyaları secret içerir.

.env dosyaları kaynak kodu ile paylaşılmamalıdır.

JWT secret production ortamında güçlü ve gizli tutulmalıdır.

SMTP password gizli tutulmalıdır.

RabbitMQ için production ortamında guest/guest kullanılmamalıdır.

Callback endpoint'i public olduğundan network erişimi sınırlandırılmalıdır.

Production ortamında HTTPS kullanılmalıdır.

Swagger endpoint'leri production ortamında erişim politikası ile sınırlandırılmalıdır.

23. Phase 1 kapsamı

Phase 1 içinde bulunan işlevler:

- JWT login
- Role tabanlı Authorization
- Admin user oluşturma
- Admin user listeleme

- Admin user silme
- İlk login password değişimi
- File upload
- Markdown conversion
- Markdown ve chunk conversion
- File status listeleme
- File reconversion
- Original file indirme endpoint'i
- Markdown indirme
- Chunk listeleme
- Tek chunk indirme
- Chunk ZIP indirme
- File silme
- Chunk silme
- Light theme
- Dark theme
- Responsive temel UI
- RabbitMQ ile asenkron processing
- PostgreSQL ile kalıcı metadata saklama

24. Phase 1 dışında kalan veya mevcut sürümde bulunmayan işlevler

Aşağıdaki işlevler kaynak kodda mevcut bir özellik olarak doğrulanmamıştır:

- Pagination
- File search
- Status filter
- UI içinde ayrıntılı per-file error görüntüleme
- UI içinde retry butonu
- Upload progress bar
- Queue progress bar
- Audit log
- Role yönetimi
- Database migration sistemi
- Otomatik end-to-end test suite
- Otomatik Frontend unit test suite

PROCESSING status enum içinde bulunur.

Mevcut upload ve callback akışında bu status değerini atayan aktif kod bulunmamaktadır.

25. Bakım kuralları

Queue, exchange ve routing key değerleri Backend ve Converter ortamlarında aynı tutulmalıdır.

Backend ve Converter aynı uctma_default network'üne bağlı olmalıdır.

Backend ve Converter aynı uctma_uploads volume'ünü kullanmalıdır.

Callback request alanları Backend FileProcessingCallbackReqDTO ile uyumlu olmalıdır.

Role değerleri aşağıdaki biçimde kullanılmalıdır:

- ROLE_USER
- ROLE_ADMIN

Conversion değerleri aşağıdaki biçimde kullanılmalıdır:

- CONVERT
- CONVERT_AND_CHUNK

Kaynak kod değişikliğinden sonra ilgili image yeniden build edilmelidir:

```
docker compose up -d --build
```

Database volume'ü silinirse PostgreSQL verileri silinir.

RabbitMQ volume'ü silinirse RabbitMQ kalıcı verileri silinir.

Uploads volume'ü silinirse original file, Markdown file ve chunk file verileri silinir.